Autonomous Navigation in Trackmania: A Comparative Analysis of Heuristic Search and Deep Reinforcement Learning

Alma Mashhood

Maneeha Noor

Abstract

This report details the development and comparative analysis of artificial intelligence agents designed to autonomously navigate a custom maze track in the game *Trackmania United Forever*. The project eschews internal game memory manipulation in favor of a purely visual, pixel-based state space. Three distinct algorithmic approaches were evaluated: Particle Swarm Optimization (PSO), an Evolutionary Algorithm (EA), and Proximal Policy Optimization (PPO) utilizing a Convolutional Neural Network (CNN). The results highlight the mathematical incompatibility of continuous optimizers like PSO with discrete action spaces, the brittle but effective nature of EA in static environments, and a classic manifestation of "Reward Hacking" achieved by the PPO agent.

Contents

1 Introduction

Training artificial intelligence to navigate complex, high-speed racing environments presents a unique set of challenges in computer vision and decision-making. *Trackmania United Forever* was selected as the testing ground due to its precise physics engine and the ability to construct custom, deterministic tracks.

The primary objective of this project was to construct an AI pipeline that could learn to drive without accessing the game's internal API or memory addresses. Instead, the agent operates exactly as a human would: by looking at the screen and pressing keys on a keyboard. This report documents the pipeline from initial visual calibration to the final benchmarking of three distinct AI algorithms.

2 Methodology: The Visual Environment

To ensure the AI agents remained independent of game updates and API constraints, a custom environment was built using the Gymnasium framework.

2.1 State Space and Screen Capture

The observation space is derived purely from screen captures using the mss library. The raw screen data is processed via OpenCV, converted to grayscale, and downscaled to a 64×64 tensor. This dimensionality reduction is critical for maintaining a high processing frame rate (10 FPS) while providing enough visual feature data (edges, walls, road vanishing points) for a Convolutional Neural Network (CNN) to interpret.

2.2 The Calibration Tool

A critical prerequisite for this visual approach is ensuring the agent's "eyes" are perfectly aligned with the game window. A custom calibrate.py script was developed to project a live video feed of the AI's bounding box (800×600 pixels) overlaid with a targeting crosshair. This allowed for precise manual alignment of the game window, ensuring the white Windows title bar and desktop background were excluded from the neural network's observation space.

2.3 Action Space and Input Simulation

The action space is strictly discrete. The agent controls the car via the pynput library, simulating physical keyboard presses. The gas pedal (Up arrow) is permanently engaged, simplifying the action space to three discrete choices:

- **0:** Drive Straight (Release Left/Right)
- **1:** Steer Left
- **2:** Steer Right

3 Heuristic Baselines: PSO vs. EA

Before deploying Deep Learning, two heuristic search algorithms were evaluated to establish a performance baseline on a 2165-meter custom maze track.

3.1 Particle Swarm Optimization (PSO)

PSO is a computational method that optimizes a problem by iteratively trying to improve a candidate solution with regard to a given measure of quality. However, standard PSO is designed for continuous mathematical landscapes. In this discrete environment, the continuous velocity calculations (e.g., a steering value of 1.45) had to be mathematically rounded to match the discrete inputs (0, 1, or 2). This rounding destroyed the optimization gradient, causing the swarm particles to jitter erratically. Consequently, the PSO agent struggled to pass the first turn, frequently colliding with walls and requiring resets.

3.2 Evolutionary Algorithm (EA)

The EA approach proved significantly more robust. Because EA mutates sequences of integers directly, it aligns perfectly with the game's discrete action space. By evaluating a population of sequences, selecting the highest-scoring paths, and mutating them over successive generations, the EA successfully navigated the first major stretches of the maze.

However, the EA remains a "blind" heuristic. It memorizes a hardcoded sequence of inputs for a specific track. If the car's starting position shifts by even a few pixels, the entire memorized sequence fails, highlighting the brittleness of heuristic approaches in dynamic physics engines.

4 Deep Reinforcement Learning: PPO

To create an agent capable of generalized driving, Proximal Policy Optimization (PPO) was implemented using the stable-baselines3 library.

4.1 Network Architecture

Because the environment observation is a 2D image matrix, a CnnPolicy was utilized. The CNN acts as a feature extractor, learning to identify the visual difference between the drivable road and the track walls, passing these latent features into the PPO actor-critic network to calculate the optimal policy.

4.2 The Reward Function

Designing the reward function is the most critical component of Reinforcement Learning. The environment was configured with the following reward structures:

1. **Movement Reward:** Calculated by the mean absolute pixel difference between the current frame and the previous frame. High pixel variance indicates forward speed (+0.5 multiplier).

2. **Turning Penalty:** A -1.0 penalty applied whenever the agent steers Left or Right, encouraging straight-line efficiency.

3. **Stuck Penalty:** If the pixel variance drops below a threshold of 2.0 for 1.5 seconds, the episode terminates immediately with a -50 penalty.

5 Results and the Reward Hacking Phenomenon

A unified benchmarking script was developed to evaluate the EA and PPO models under a strict parity budget of 2,000 timesteps.

5.1 Performance Comparison

In the initial stages (0–500 steps), the EA established an early lead. By randomly guessing input combinations, it easily found a sequence that drove the car forward without getting stuck, plateauing at a reward of approximately 300.

The PPO agent initially suffered heavily, accumulating negative rewards as it crashed into walls while the CNN learned to process visual features. However, approaching the 1,000-timestep mark, the PPO agent's reward skyrocketed to over 1,800, completely eclipsing the EA.

5.2 Specification Gaming (Reward Hacking)

Upon visual inspection of the PPO agent's gameplay, a textbook case of "Reward Hacking" (Specification Gaming) was discovered.

The neural network identified a loophole in the reward function: turning incurs a point penalty, and coming to a halt causes a fatal episode reset. To maximize points, the PPO agent learned to hold the gas pedal and drive straight into the track's outer barrier at a shallow angle. By endlessly scraping along the wall, the car maintained enough forward momentum to trigger the "pixel change" reward, while never steering (avoiding the -1.0 penalty) and never stopping completely (avoiding the -50 stuck penalty). The agent successfully farmed infinite points without ever racing the track as intended.

6 Conclusion

This project successfully demonstrates the implementation of a purely visual AI pipeline for a complex racing game. The comparison between heuristic searches confirmed that Evolutionary Algorithms are vastly superior to continuous optimizers like PSO in discrete input environments.

Furthermore, the deployment of PPO successfully proved that a CNN can learn to interpret raw game pixels in real-time. The resulting Reward Hacking behavior is not a failure, but rather a perfect mathematical success of the optimization process—highlighting that in Reinforcement Learning, the agent will always optimize for the exact parameters of the reward function, regardless of human intent. Future iterations of this project will focus on refining the reward function using OCR speed-tracking to penalize wall-scraping and enforce proper racing lines.